Quiz 3 (Version 1)

CS 320: Concepts of Programming Languages

October 28, 2026

Name: Nathan Mull

BUID: 12345678

- ▷ You will have approximately 35 minutes to complete this quiz. Make sure to read every question; some are easier than others.
- ▷ Make it very clear what your final solution for each problem is (e.g., by surrounding it in a box). We reserve the right to mark off points if we cannot tell what your final solution is.
- ▷ You must show your work on all problems unless otherwise specified. A solution without work will be considered incorrect (and will be investigated for potential academic misconduct).

1 OCaml Programming

(10 points) Implement the function

```
val map2_filter : ('a -> 'b -> 'c option) -> 'a list -> 'b list -> 'c list
```

so that `map2_filter f l r` is the list of values `v` such that `f` applied to corresponding elements of `l` and `r` is `Some v`, in order. If `l` and `r` have different lengths, the extra elements of the longer list are ignored.

```
let f x y = let sum = x + y in if sum mod 2 = 0 then Some sum else None
let _ = assert(map2_filter f [1;2;3;1] [2;4;6;3;4] = [2 + 4; 1 + 3]) (* 2nd, 4th *)
let _ = assert(map2_filter f [1;3;5] [1;9] = [2;12]) (* 1st, 2nd *)
let _ = assert(map2_filter f [1;2] [2;3] = [])
let _ = assert(map2_filter f [] [1;2;3] = [])

let g x b = if b then Some x else None
let _ = assert(map2_filter g ["a";"b";"c"] [true;false;true] = ["a";"c"])
let _ = assert(map2_filter g ["a";"b";"c"] [false;true] = ["b"])
```

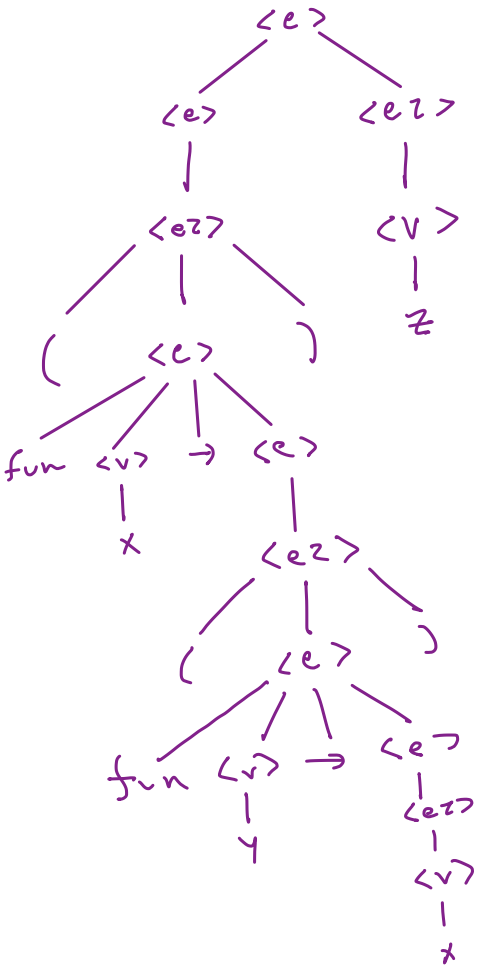
```
let map2_filter f l r =
    let rec go l r =
        match l, r with
        | [], [] -> []
        | [], _ -> []
        | _, [] -> []
        | x :: l, y :: r ->
            match f x y with
            | Some a -> a :: go l r
            | None -> go l r
    in go l r
```

2 Formal Grammar

Consider the following BNF grammar.

```
<e>  ::= fun <v> -> <e> | <e> <e2> | <e2>
<e2> ::= <v> | (<e>)
<v>  ::= x | y | z
```

A. (10 points) Demonstrate that the sentence `(fun x -> (fun y -> x)) z` is recognized by the above grammar by providing a parse tree for it.



The grammar from the previous part is reproduced for convenience.

```
<e>  ::= fun <v> -> <e> | <e> <e2> | <e2>
<e2> ::= <v> | (<e>)
<v>  ::= x | y | z
```

B. (10 points) Demonstrate that the above grammar is ambiguous by determining a sentence with multiple parse trees. Include both parse trees in your answer.

fun x → x x

